Backpressure in Messaging

when producers outrun consumers

Phase 4 videos on YouTube

In one line: a broker absorbs bursts as lag, but sustained overload needs a response: scale consumers (up to the partition count), slow producers with quotas or bounded buffers, or shed low-value work, while watching lag and protecting the systems your consumers write to.

1 Black Friday

Orders pour in at ten times the normal rate and your consumers can't keep up. Messages pile up in the broker, minute after minute. Nothing has crashed, but confirmation emails are now 40 minutes late, and the backlog keeps growing. This is the messaging version of **backpressure**: the downstream signalling, or being made to signal, 'slow down'.

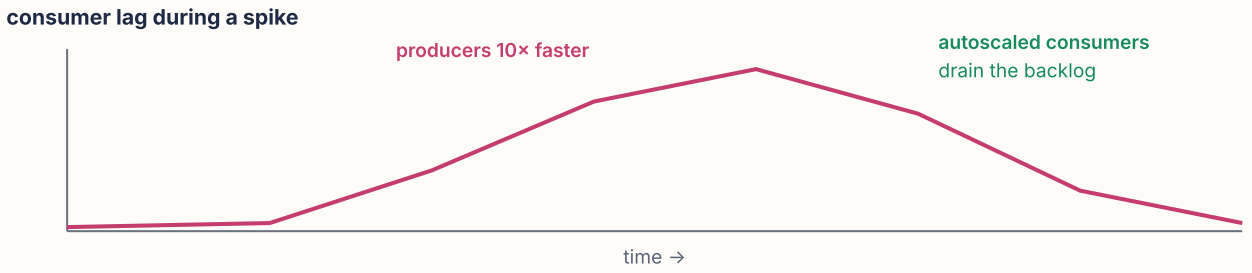


Figure 1. Lag grows while producers outrun consumers, and shrinks when consumers catch up.

2 The broker is a buffer, not a solution

Decoupling producers from consumers is the point of a broker (lesson 23): a short spike simply becomes **lag** and drains afterwards. But if the input rate stays above the processing rate, lag grows without limit: messages get old, may hit retention limits and be deleted, and catching up takes hours. Measure **lag in messages and in time** (age of the oldest unprocessed message) and alert on both.

3 Ways to respond

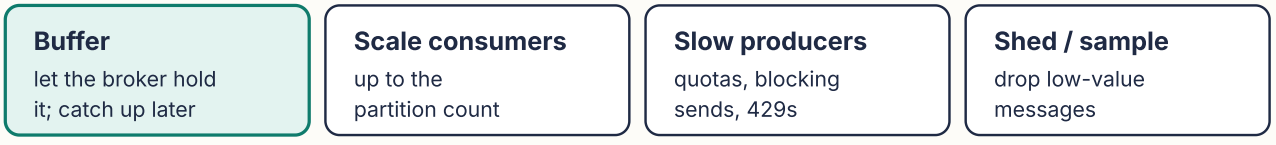


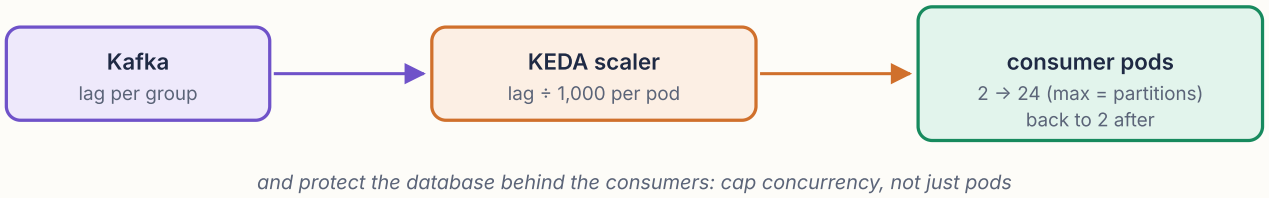
Figure 2. The four responses to sustained overload. Usually you combine the first two.

- **Scale consumers** on lag, up to the partition count, and make each consumer faster (batching, async I/O).
- **Slow producers:** Kafka quotas throttle clients; a full bounded producer buffer blocks `send()`; an API in front can return 429. With RabbitMQ, queue length limits and flow control push back on publishers.
- **Shed or sample:** drop debug events or analytics samples first; never orders.

- **Prioritise:** separate topics or queues for critical and bulk work, so bulk can't starve critical.

WORKED EXAMPLE: KEDA SCALING CONSUMERS ON KAFKA LAG

KEDA is an event-driven autoscaler for Kubernetes. You give it a Kafka scaler with the consumer group, a target of 1,000 messages of lag per pod, and a maximum of 24 pods, the topic's partition count. When the spike hits, lag climbs past 20,000 and KEDA scales from 2 to 24 pods within a minute or two; when lag drains, it scales back. Because 24 pods would overload the orders database, each pod caps its concurrent writes and uses a shared connection pool limit, so the backlog drains at the rate the database can take.



and protect the database behind the consumers: cap concurrency, not just pods

Figure 3. Lag-based autoscaling, bounded by partitions and by what downstream can handle.

COMMON MISTAKES

- Scaling on CPU instead of lag.
- Scaling consumers beyond the partition count.
- Unbounded in-memory queues inside consumers, which run out of memory.
- Letting retention delete unprocessed messages during a long backlog.

4 Key terms

Backpressure: signal from a slow consumer that slows the producer

Quota: broker-enforced rate limit per client

KEDA: Kubernetes autoscaler driven by event sources like Kafka lag

Consumer lag: messages (or time) the consumer is behind

Load shedding: dropping low-value work under overload

5 Interview questions

Q1. How does a message broker help with traffic spikes? BASIC

It buffers messages so producers aren't slowed; consumers work at their own pace and catch up later, as lag.

Q2. Which metric should drive consumer autoscaling? BASIC

Consumer lag, ideally both message count and age of the oldest message, rather than CPU.

Q3. Lag keeps growing for hours. What do you do? SCENARIO

Find the bottleneck (consumer CPU, downstream DB, partition count). Scale consumers up to the partitions, add partitions if needed, batch writes, and throttle or shed low-priority producers.

Q4. Scaling consumers overloaded the database. Fix it. SCENARIO

Limit concurrency per consumer and in total, use bulk writes, and let lag absorb the rest; the database's capacity is the real limit.

Q5. How can a Kafka producer feel backpressure? TRICKY

Its buffer fills when brokers are slow or throttled by quotas; send() then blocks up to max.block.ms or fails, which you can surface as 429s upstream.

Q6. When is dropping messages acceptable? TRICKY

For low-value or aggregatable data like metrics, logs and analytics samples, never for orders or payments; make it explicit and measured.

Next: SD-I-012 · Event Types: Notifications, State Transfer, Commands backendroadmap.com